`scripts/run-challenge-matrix.sh` — User Guide

Last verified: 2026-08-26 (LVA-161 — `--test-timeout` pass-through; the CLI's 10m default was killing whole-module sweeps and destroying every class's results) **Inheritance:** HelixConstitution §11.4.18 + Lava §6.AE.6 + §6.X (Container-Submodule Emulator Wiring) + §6.I (Multi-Emulator Container Matrix)

Overview

Operator entry point for §6.AE gate-mode Challenge Test runs. Pre-bakes the §6.AE.2 minimum AVD matrix (API 28 / 30 / 34 / latest stable × phone + tablet) and delegates to `submodules/containers/cmd/emulator-matrix --runner=containerized` per §6.X.

LVA-014 durable device-gate fixes (2026-07-26)

The 2026-07-04 "device gate boot hang" was diagnosed as an **AVD-name mismatch**: the baked emulator images (`ghcr.io/vasic-digital/lava-android-emulator:apiNN-x86_64`) bake exactly ONE AVD named `default`, but the runner passed the §6.AE.2 matrix names (`CZ_API34_Phone`, …) verbatim. The image entrypoint exited in ~4s ("AVD not found"), the container's `--rm` reaped the log, and `WaitForBoot` polled the dead forwarded port to its deadline, misreporting the 4s exit as a boot timeout. Three durable fixes:

1. **AVD-name resolution** (`submodules/containers/pkg/emulator/avdresolve.go`, wired into `Containerized.Boot`) — the runner queries the AVDs actually baked into each image (`avdmanager list avd` inside a throwaway `--rm --entrypoint avdmanager` container) and maps the requested `name:api:form` entry to a real baked AVD. Exact name match → used verbatim (the proven `--avds "default:34:phone"` workaround is byte-compatible). Api-level match → the baked name is substituted with a stderr note (the requested name was advisory). No match → Boot fails FAST, before any container is launched, naming the available baked AVDs.

2. **Container liveness in WaitForBoot** (`submodules/containers/pkg/emulator/containerized.go`) — every poll iteration first checks `podman/docker inspect` that the emulator container is still running; on exit it fails immediately with the container's captured logs (`<runtime> logs --tail 100`, fetched before `--rm` reaps them; an already-reaped container is reported honestly). EMU-1 semantics are preserved: a `Containerized` without a Boot-set container name performs no liveness exec, and every liveness exec is bound to the WaitForBoot deadline context.
3. **Emulator-image preflight** (this script, containerized runner only) — before building the APK, the script instantiates the `{api}` image template per distinct api level in the matrix, checks each image with `<runtime> image inspect`, and pulls the missing ones with live progress. A pull failure is an honest operator-facing error (exit 1) that names the image, shows the pull error tail, and prints the local-build fallback command from `submodules/containers/pkg/emulator/Containerfile` — never a silent skip.

The default `--container-image` is now the template `ghcr.io/vasic-digital/lava-android-emulator:api{api}-x86_64`; the runner substitutes `{api}` with each AVD's api level at Boot time, so one reference covers the whole multi-api matrix. A `--container-image` without `{api}` is used verbatim for every AVD (the pre-LVA-014 single-image behavior).

Honest pre-flight

The script DETECTS the §6.X-debt darwin/arm64 host gap (no `/dev/kvm` available to podman containers; macOS HVF not exposed to podman) and, when detected, REFUSES to claim a successful gate run. Instead it:

1. Validates arguments + matrix minimum
2. Writes `<evidence-dir>/host-preflight.json` with the host-gap classification
3. Exits 2 (NOT 0) — gate-host ineligible

Per §6.J/§6.L: no false-pass; honest unblock report. Real gate runs require a Linux x86_64 + KVM host.

Usage

```
# Run ALL Challenges on the §6.AE.2 minimum matrix
bash scripts/run-challenge-matrix.sh

# Run a specific Challenge
bash scripts/run-challenge-matrix.sh \
  --test-class
        lava.app.challenges.Challenge01AppLaunchAndTrackerSelectionTest
```

```
# Add TV-class AVD (when feature touches TvActivity)
bash scripts/run-challenge-matrix.sh --add-tv

# Add foldable AVD
bash scripts/run-challenge-matrix.sh --add-foldable

# Override "latest stable" API level
bash scripts/run-challenge-matrix.sh --latest-api 36

# Skip APK rebuild
bash scripts/run-challenge-matrix.sh --no-build

# Override the matrix entirely with explicit AVDs (target EXISTING
        host AVDs).
# Use on a macOS host-direct+HVF run where the default CZ_API*
        images are not provisioned.
bash scripts/run-challenge-matrix.sh --avds
        "CZ_API35_Phone_Fresh:35:phone" --no-build

# Single-AVD smoke run against the locally present api34 image
        (the requested
# name is advisory — the runner resolves the image's baked AVD,
        e.g. "default")
bash scripts/run-challenge-matrix.sh --avds
        "CZ_API34_Phone:34:phone" --no-build \
  --test-class lava.app.challenges.Challenge00ColdStartTest

# Pin one explicit image for every AVD (pre-LVA-014 single-image
        behavior)
bash scripts/run-challenge-matrix.sh \
  --container-image ghcr.io/vasic-digital/lava-android-
        emulator:api34-x86_64

# ALSO invoke the 11 HelixQA Challenge scripts (Option 1 wiring)
bash scripts/run-challenge-matrix.sh --include-helixqa
```

Inputs

Arg	Description
`--test-class <fqn>`	Specific Challenge to run (default: all under `lava.app.challenges`)
`--evidence-dir <dir>`	Output directory (default: dated under `.lava-ci-evidence/`)
`--no-build`	Skip the `:app:assembleDebug` step
`--avds "<name:api:form[,...]>"`	**Replace** the §6.AE.2 default matrix entirely with an explicit comma-separated AVD list. Use to target **existing host AVDs** (e.g. `CZ_API35_Phone_Fresh:35:phone` on a

Arg	Description
	macOS host-direct+HVF run where the default `CZ_API*` images are not provisioned). When set, `--latest-api` / `--add-tv` / `--add-foldable` are ignored. A sub-minimum list is a **development-iteration** run, **not** a §6.AE.2-conformant gate matrix. NOTE: the Containers CLI still resolves each AVD's system image through `tools/lava-containers/vm-images.json` (`--image-manifest`); the manifest must contain a matching image id (e.g. `android-35-phone`) or the runner fails with `no image with id`.
`--latest-api <N>`	Override the "latest stable" API level (default: 36; ignored when `--avds` is set)
`--container-image <ref>`	Containerized-runner image. Default: `ghcr.io/vasic-digital/lava-android-emulator:api{api}-x86_64` — the `{api}` token is substituted per AVD api level by the runner (LVA-014 fix #1). A ref without `{api}` is used verbatim for every AVD.
`--container-runtime <podman|docker>`	Container runtime CLI (default: podman).
`--extra-apk <path>`	Additional APK installed on each AVD AFTER the client APK, before instrumentation runs. **Repeatable** — pass more than once to install several companion APKs. Forwarded verbatim, in order, to the Containers CLI's own repeatable `--extra-apk` flag (`MatrixConfig.ExtraAPKPaths`). Use for Challenges needing a second app on the SAME device (e.g. Challenge72's on-device api-app companion for same-device mDNS discovery). Added for the §6.X gate wiring that closes the two-app-on-one-emulator capability gap.
`--add-tv`	Add a TV-class AVD to the matrix
`--add-foldable`	Add a foldable AVD
`--include-helixqa`	ALSO invoke `scripts/run-helixqa-challenges.sh` (the 11 HelixQA Challenge scripts per Option 1 wiring). OFF by default so existing matrix runs are unaffected. HelixQA runs on the HOST BEFORE the AVD matrix →

Arg	Description
	independent of §6.X-debt darwin/arm64 gate-host gap. HelixQA wrapper evidence lands at `<evidence-dir>/helixqa/`. Non-zero HelixQA exit promotes the final aggregate exit code (matrix exit dominates if both fail). See `docs/scripts/run-helixqa-challenges.sh.md` for full details.

LVA-161 — `--test-timeout` pass-through (2026-08-26)

The defect

`submodules/containers/cmd/emulator-matrix` defaults `--test-timeout` to **10 minutes** (`cmd/emulator-matrix/main.go:123`). That budget covers the **TEST step only** — cold boot is timed separately and reported as `boot_seconds` — and this script never set it, so every run inherited the default.

A whole-module Challenge sweep is a **single gradle invocation** covering every selected class, so 10 minutes is far below the real workload. Measured: the `2026-08-26T14-09-17Z` run was killed at `test_seconds=600.02` with `signal: killed`, having completed **81 of 104 tests**.

What makes this worse than a truncated run: gradle writes its JUnit XML at the **end** of the invocation. A kill therefore destroys the results of **every** class in the sweep, including the ones that had already passed — so the matrix reports the whole selection as failed when nothing about the product was wrong. That is a verdict determined by a clock, not by the code under test.

What changed here

This script gained a `--test-timeout <duration>` argument that is forwarded verbatim to `emulator-matrix --test-timeout` when set, and omitted entirely when empty (preserving the CLI default). That is the whole change: this script does **not** compute a value.

Callers MUST size it to the selected workload. The pipeline's Challenge phase (`scripts/pipeline/phase-02-test-challenge.sh`) derives it as `300s + 45s × <selected class count>` — a fixed overhead plus a per-class budget — and prints the derivation with the class count before invoking. Both terms are overridable (`LAVA_PIPELINE_CHALLENGE_TEST_TIMEOUT_OVERHEAD_S`, `LAVA_PIPELINE_CHALLENGE_TEST_TIMEOUT_PER_CLASS_S`), and an explicit

`LAVA_PIPELINE_CHALLENGE_TEST_TIMEOUT` wins verbatim. Operators invoking this script directly for a large selection should pass `--test-timeout` themselves rather than accept 10m.

Reading a timeout kill honestly

A run killed at the timeout is **not** a test failure and must not be recorded as one. The distinguishing evidence is `test_seconds` sitting within a rounding error of the configured budget together with `signal: killed`, and a JUnit XML that is absent rather than reporting failures. The remedy is to raise `--test-timeout` (or reduce the class selection) and re-run to obtain a real verdict — never to re-interpret the kill as a product defect.

Inputs (addition)

Arg	Description
`--test-timeout <duration>`	Forwarded to `emulator-matrix --test-timeout` (Go duration syntax, e.g. `2400s`, `40m`). Empty = the CLI default of 10m, which is **too small for a whole-module sweep**. Applies to the TEST step only; cold boot is budgeted separately by `--boot-timeout`.

§6.AE.2 mandatory minimum matrix

```
CZ_API28_Phone:28:phone
CZ_API30_Phone:30:phone
CZ_API34_Phone:34:phone
CZ_API34_Tablet:34:tablet
CZ_API<latest>_Phone:<latest>:phone
```

Plus TV / foldable when explicitly requested by `--add-tv` / `--add-foldable`.

Sub-minimums are permitted for development iteration; the gate row's `gating: true` flag is only set when the full minimum is satisfied + every config dimension (theme/locale/density per §6.AE.2) is covered.

Outputs

- `<evidence-dir>/host-preflight.json` — host-gap classification (always written)
- `<evidence-dir>/real-device-verification.{md,json}` — per-AVD attestation rows (only on Linux x86_64 + KVM gate-host)

- `<evidence-dir>/helixqa/helixqa-attestation.json` + per-script logs — only when `--include-helixqa` is set

Exit codes

Code	Meaning
0	Matrix completed; all AVDs passed
1	At least one AVD failed boot OR at least one test failed (real Containers CLI exit), OR the LVA-014 image preflight failed (missing image + pull failure — honest error with local-build fallback)
2	Gate-host ineligible (darwin/arm64 OR no KVM); honest unblock-needed report written

Cross-references

- `submodules/containers/cmd/emulator-matrix` (the underlying runner)
- `tools/lava-containers/vm-images.json` (matrix manifest)
- `scripts/run-emulator-tests.sh` (older sister-glue; same delegation, different default arguments)
- `scripts/run-helixqa-challenges.sh` + `docs/scripts/run-helixqa-challenges.sh.md` (the HelixQA wrapper invoked by `--include-helixqa`)
- `docs/plans/2026-05-16-helixqa-integration-design.md` (Option 1 wiring design)
- `.lava-ci-evidence/sixth-law-incidents/2026-05-13-emulator-container-darwin-arm64-gap.json` (the standing §6.X-debt incident)
- Lava `CLAUDE.md` §6.AE + §6.X + §6.I